

1. Enable CloudTrail monitoring and store the events in S3 and CloudWatch log events.

Objective

To configure AWS CloudTrail to monitor AWS account activity and store CloudTrail events in both an Amazon S3 bucket and Amazon CloudWatch Logs.

Resources Used

Resource	Name
CloudTrail Trail	devops-cloudtrail-asif-2026
S3 Bucket	devops-cloudtrail-logs-asif-2026
CloudWatch Log Group	/aws/cloudtrail/devops-cloudtrail-asif-2026
IAM Role	CloudTrail-CloudWatchLogs-Role-asif-2026
AWS Region	ap-south-1

Implementation Steps

Step 1: Create/Configure CloudTrail Trail

1. Open the AWS Management Console.
2. Navigate to CloudTrail → Trails.
3. Created/configured the trail with the name:
devops-cloudtrail-asif-2026
4. Enabled Multi-region trail.
5. Selected the existing S3 bucket:
devops-cloudtrail-logs-asif-2026
6. S3 log file validation was enabled.
7. SSE-KMS encryption was kept disabled because customer-managed KMS encryption was not required for this assignment.

Step 2: Configure CloudWatch Logs

1. Enabled CloudWatch Logs for the CloudTrail trail.
2. Configured the Log Group:
/aws/cloudtrail/devops-cloudtrail-asif-2026

3. Configured the IAM role:
CloudTrail-CloudWatchLogs-Role-asif-2026
4. This allows CloudTrail to deliver events to CloudWatch Logs.

Step 3: Configure Management Events

1. Opened Step 2 – Choose log events.
2. Enabled Management Events.
3. Selected both:
 - a. Read events
 - b. Write events
4. This allows CloudTrail to record management/API activity in the AWS account.

Step 4: Verify CloudTrail Logs in S3

1. Navigated to Amazon S3.
2. Opened the bucket:
devops-cloudtrail-logs-asif-2026
3. Navigated through:

AWSLogs → 331116047691 → CloudTrail → ap-south-1 → 2026 → 09 → 10

4. Verified that CloudTrail generated a compressed log file:

331116047691_CloudTrail_ap-south-1_20260910T0800Z_YkL6Vk9gga1UjpfD.json.gz

5. The presence of this file confirms that CloudTrail events are being successfully stored in S3.

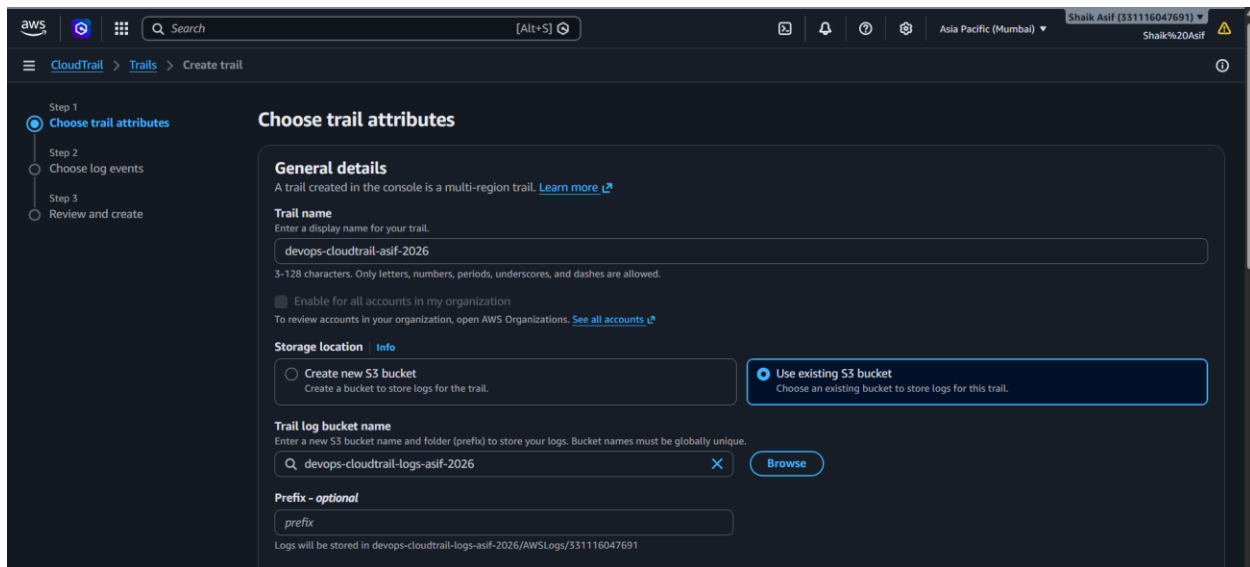
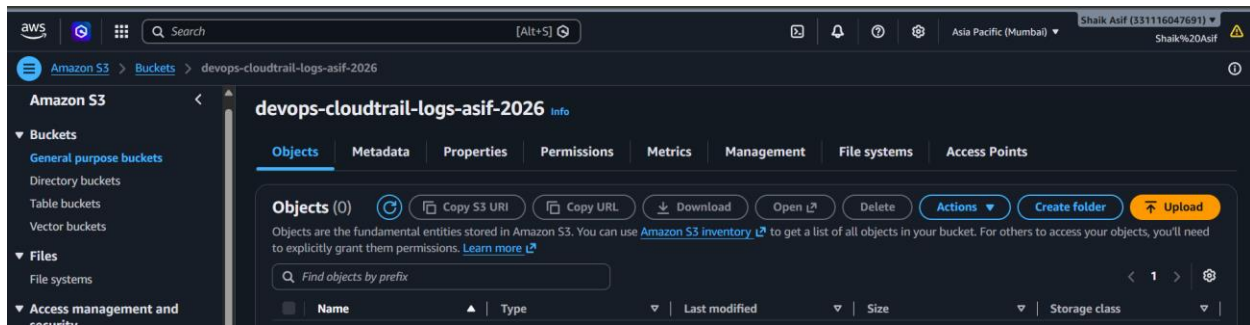
Step 5: Verify CloudTrail Logs in CloudWatch

1. Navigated to CloudWatch → Logs → Log Management → Log groups.
2. Opened:

/aws/cloudtrail/devops-cloudtrail-asif-2026

3. Opened the available log stream.
4. Verified an actual CloudTrail event in the log stream.
5. The event contained fields including:
 - a. eventTime
 - b. eventSource
 - c. eventName

- d. awsRegion
 - e. managementEvent
 - f. eventCategory
6. The verified event was an AssumeRole management event generated by the CloudTrail service for CloudWatch Logs delivery.



aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

CloudTrail > Trails > Create trail

SNS notification delivery

Info

☐ Enabled

CloudWatch Logs - optional

Configure CloudWatch Logs to monitor your trail logs and notify you when specific activity occurs. Standard CloudWatch and CloudWatch Logs charges apply. [Learn more](#)

CloudWatch Logs

Info

☒ Enabled

Log group

Info

☒ New

☐ Existing

Log group name

1-512 characters. Only letters, numbers, dashes, underscores, forward slashes, and periods are allowed.

IAM Role

Info

☒ New

☐ Existing

Role name

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

CloudTrail > Trails > Create trail

Step 2

☒ Choose log events

Step 3

☐ Review and create

Events

Info

Record API activity for individual resources, or for all current and future resources in AWS account. [Additional charges apply](#)

Event type

Choose the type of events that you want to log.

☒ Management events

Capture management operations performed on your AWS resources.

☐ Data events

Log the resource operations performed on or within a resource.

☐ Insights events

Identify unusual activity, errors, or user behavior in your account.

☐ Network activity events

Network activity events provide information about resource operations performed on a resource within a virtual private cloud endpoint.

Management events

Info

Management events show information about management operations performed on resources in your AWS account.

☒ No additional charges apply to log management events on this trail because this is your first copy of management events.

API activity

Choose the activities you want to log.

☒ Read

☒ Write

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

CloudTrail > Trails

Copy events to Lake

Refresh

Delete

Create trail

	Name	Home region	Multi-region trail	ARN	Insights	Organization trail	S3 bucket	Log file prefix	CloudWatch Logs group
☐	devops-cloudtrail-asif-2026	Asia Pacific (Mumbai)	Yes	arn:aws:cloudtrail:ap-south-1:331116047691:trail/devops-cloudtrail-asif-2026	Disabled	No	devops-cloudtrail-logs-asif-2026	-	arn:aws:logs:ap-south-1:331116047691:group/aws/cloudtrail/devops-cloudtrail-asif-2026:

Amazon S3 > Buckets > devops-cloudtrail-logs-asif-2026 > AWSLogs/ > 331116047691/ > CloudTrail/ > ap-south-1/ > 2026/ > 09/ > 10/

10/

Copy S3 URI

Objects Properties

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
331116047691_CloudTrail_ap-south-1_20260910T0800Z_YkLGvK9gga1UjpfD.json.gz	gz	September 10, 2026, 13:28:14 (UTC+05:30)	4.8 KB	Standard

CloudWatch > Log management > /aws/cloudtrail/devops-cloudtrail-asif-2026

CloudWatch

Retention: Never expire

Deletion protection: Off

Stored bytes: -

Data protection: -

Account: -

Not supported

Bearer token authentication: Off

Syslog ingestion: [Configure](#) [Info](#)

Log streams (4)

By default, we only load the most recent log streams.

Filter log streams or try prefix search

Exact match Show expired

Log stream	Last event time
331116047691_CloudTrail_ap-south-1_2	2026-09-10 07:59:17 (UTC)
331116047691_CloudTrail_ap-south-1	2026-09-10 07:58:56 (UTC)
331116047691_CloudTrail_ap-south-1_4	2026-09-10 07:58:22 (UTC)
331116047691_CloudTrail_ap-south-1_3	2026-09-10 07:57:07 (UTC)

CloudWatch > Log management > /aws/cloudtrail/devops-cloudtrail-asif-2026 > 331116047691_CloudTrail_ap-south-1_2

CloudWatch

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom UTC timezone

Display

Timestamp	Message
No older events at this moment. Retry	
2026-09-10T07:59:17.603Z	{"eventVersion":"1.11","userIdentity":{"type":"AWSService","invokedBy":"cloudtrail.amazonaws.com"},"eventTime":"2026-09-10T07:57:07Z"...
No newer events at this moment. Auto retry paused . Resume	

Result

CloudTrail monitoring was successfully enabled. CloudTrail events are being delivered and stored in the configured Amazon S3 bucket and are also being sent to the configured Amazon CloudWatch Log Group.

Conclusion

The CloudTrail monitoring setup was successfully completed and verified. The generated CloudTrail log file was confirmed in S3, and an actual CloudTrail management event was confirmed in CloudWatch Logs. Therefore, AWS account activity can now be monitored and retained through both S3 and CloudWatch.

2. Enable SNS for CloudTrail to send alerts via email.

Objective

To configure Amazon SNS with CloudTrail so that CloudTrail notifications can be sent to a registered email address.

Resources Used

Resource	Name
AWS Region	Mumbai (ap-south-1)
CloudTrail Trail	devops-cloudtrail-asif-2026
SNS Topic	devops-cloudtrail-alerts-asif-2026
Protocol	Email

Implementation Steps

Step 1: Open Amazon SNS

1. Logged in to the AWS Management Console.
2. Opened Amazon SNS.
3. Selected the Mumbai (ap-south-1) region.

Step 2: Create SNS Topic

1. Navigated to SNS → Topics.
2. Clicked Create topic.
3. Selected Standard topic type.
4. Entered the topic name:
devops-cloudtrail-alerts-asif-2026
5. Created the SNS topic.

Step 3: Create Email Subscription

1. Opened the created SNS topic.
2. Clicked Create subscription.
3. Selected:
 - a. Protocol: Email
 - b. Endpoint: Registered email address
4. Created the subscription.

Step 4: Confirm Email Subscription

1. Received the AWS SNS confirmation email.
2. Opened the email.
3. Clicked the Confirm subscription link.
4. Verified that the subscription status changed to Confirmed.

Step 5: Configure CloudTrail SNS Notification

1. Opened AWS CloudTrail.
2. Navigated to Trails.
3. Selected:
devops-cloudtrail-asif-2026
4. Edited the trail configuration.
5. Enabled SNS notification delivery.
6. Selected the SNS topic:
devops-cloudtrail-alerts-asif-2026
7. Saved the changes.

Step 6: Verify Configuration

Verified that:

- SNS topic was created successfully.
- Email subscription was Confirmed.
- CloudTrail SNS notification delivery was enabled.
- The CloudTrail trail was associated with the SNS topic.

Create topic

Details

Type [Info](#)
Topic type cannot be modified after topic is created

☐ FIFO (first-in, first-out)

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ Standard

- Best-effort message ordering
- At-least once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

Name

devops-cloudtrail-alerts-asif-2026

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

Display name - optional [Info](#)
To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

My Topic

Maximum 100 characters.

► **Encryption - optional**

Amazon SNS provides in-transit encryption by default. Enabling server-side encryption adds at-rest encryption to your topic.

Create subscription

Details

Topic ARN

arn:aws:sns:ap-south-1:331116047691:devops-cloudtrail-alerts-asif-2026

Protocol
The type of endpoint to subscribe

Email

Endpoint
An email address that can receive notifications from Amazon SNS.

asifshaik6558@gmail.com

After your subscription is created, you must confirm it. [Info](#)



Simple Notification Service

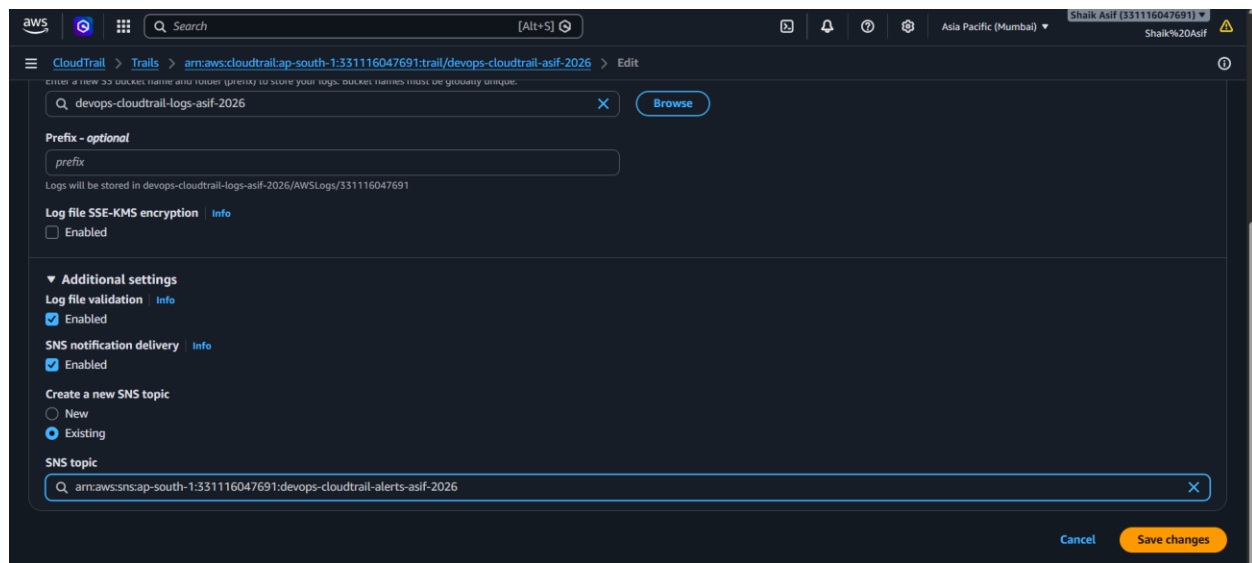
Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:ap-south-1:331116047691:devops-cloudtrail-alerts-asif-2026:cbb4ffcc-047a-4056-bf5c-e7da19264e7f

If it was not your intention to subscribe, [click here to unsubscribe](#).



Result

SNS was successfully configured with CloudTrail, and the email subscription was confirmed. CloudTrail is now configured to publish its notifications through the SNS topic.

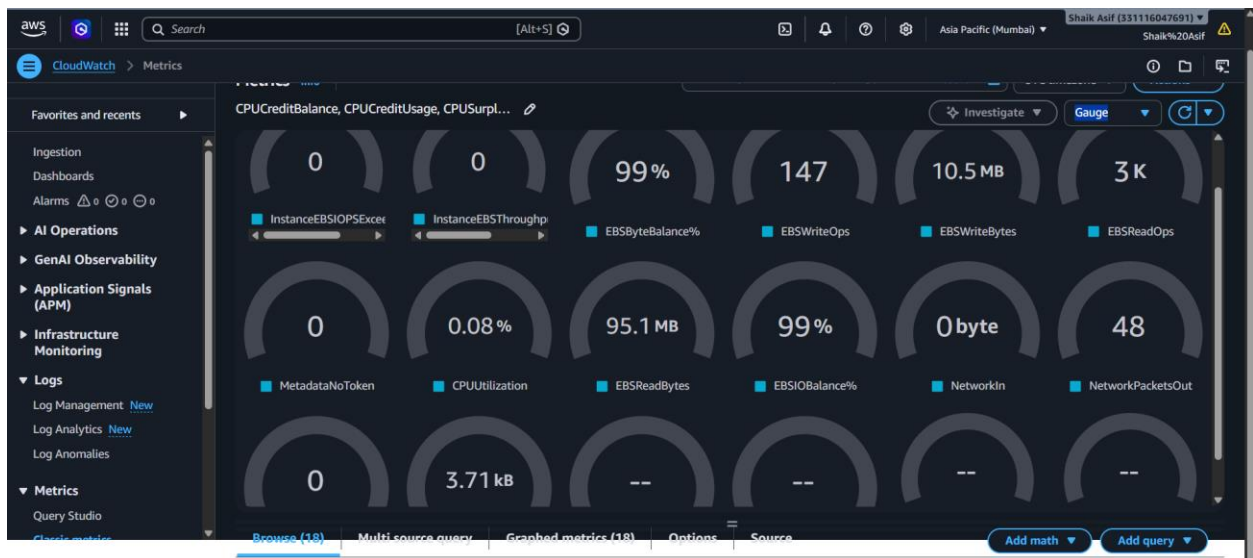
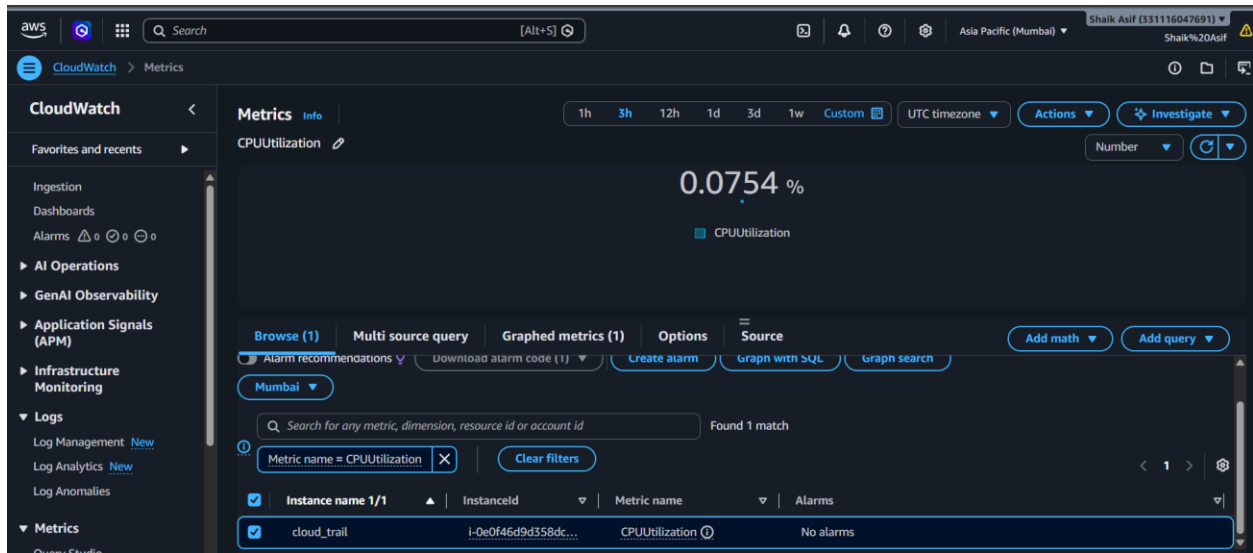
Conclusion

Successfully completed the integration of CloudTrail with Amazon SNS and email notifications. This configuration enables CloudTrail-related notification messages to be delivered to the registered email address through SNS.

3 Configure CloudWatch monitoring and record the CPU utilization and other metrics of EC2.

Steps

1. Logged in to the **AWS Management Console**.
2. Opened **Amazon CloudWatch**.
3. Selected the **Mumbai (ap-south-1)** region.
4. From the left navigation menu, selected **Metrics**.
5. Clicked **All metrics**.
6. Selected the **EC2** service.
7. Selected **Per-Instance Metrics**.
8. Selected the required EC2 instance.
9. Verified the **CPUUtilization** metric and viewed its graph.
10. Verified other available EC2 metrics, including:
 - CPUUtilization
 - NetworkIn
 - NetworkOut
 - NetworkPacketsIn
 - NetworkPacketsOut
 - EBSReadBytes
 - EBSReadOps
 - EBSWriteBytes
 - EBSWriteOps
 - CPU credit metrics
 - EC2 status check metrics
11. Verified that CloudWatch was receiving and displaying metric data for the EC2 instance.
12. Captured the CloudWatch metrics and CPU utilization graph as evidence.



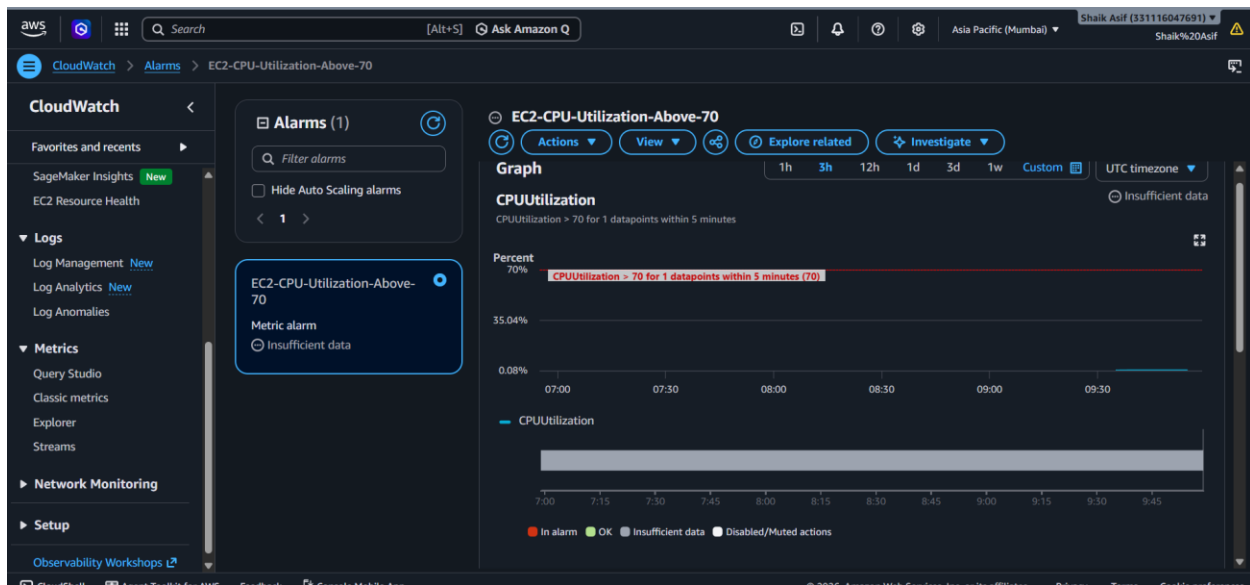
Conclusion

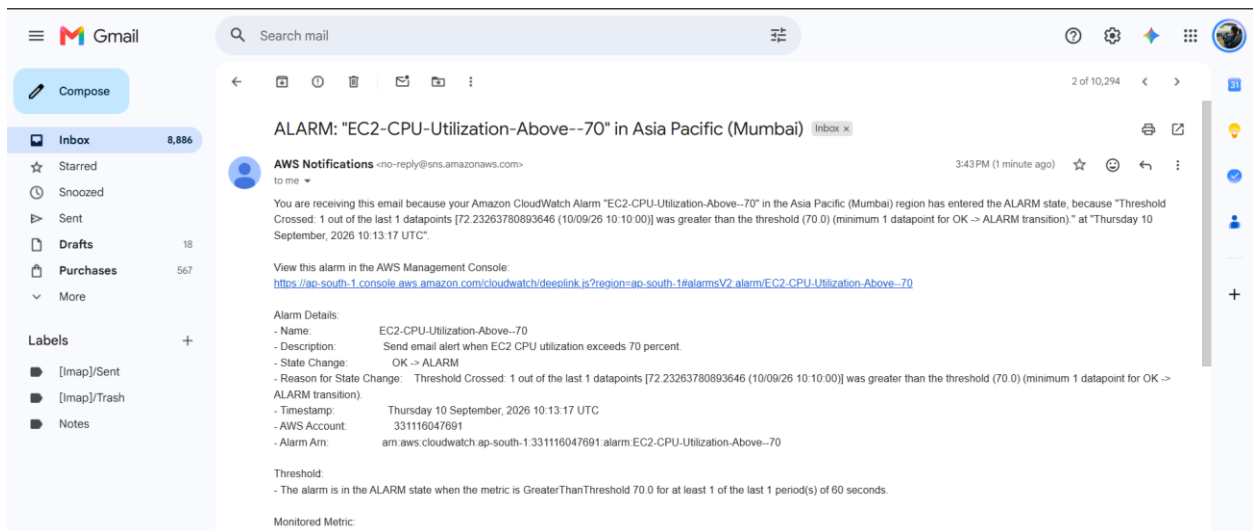
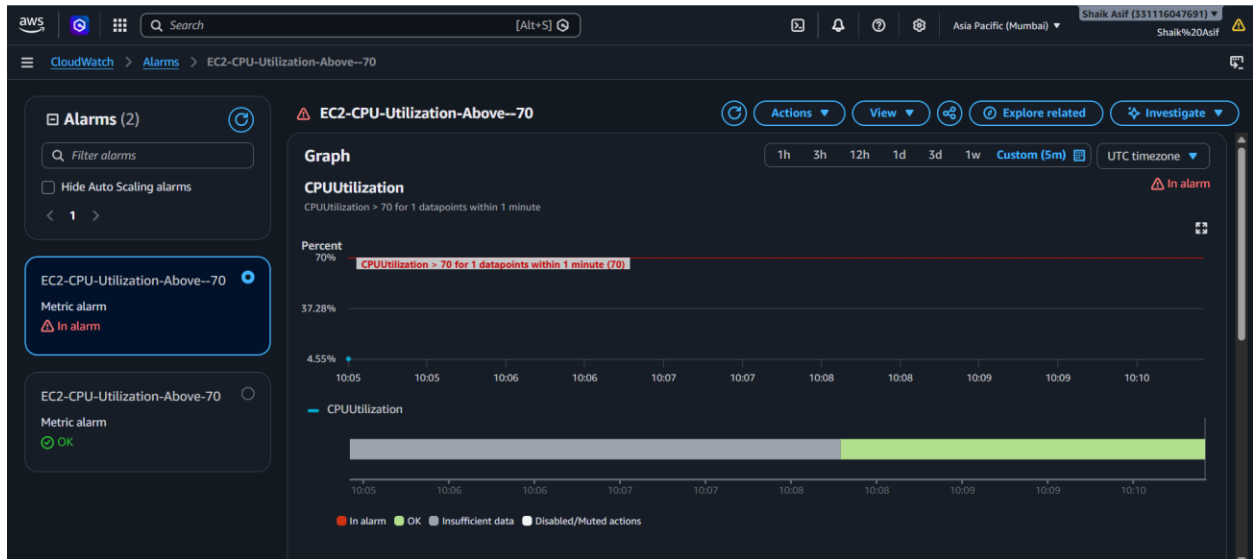
Successfully configured Amazon CloudWatch monitoring for the EC2 instance and verified CPU utilization and other EC2 performance and health metrics. The metrics are available in CloudWatch for monitoring and can be used for creating alarms and dashboards.

4. Create one alarm to send an alert to email if the CPU utilization is more than 70 percent.

Steps

1. Opened **Amazon CloudWatch** in the AWS Management Console.
2. Navigated to **Metrics** → **All metrics** → **EC2** → **Per-Instance Metrics**.
3. Selected the required EC2 instance.
4. Selected the **CPUUtilization** metric.
5. Clicked **Actions** → **Create alarm**.
6. Configured the alarm condition as:
 - a. **Statistic:** Average
 - b. **Period:** 5 minutes
 - c. **Threshold:** Greater than **70%**
7. Configured the **SNS notification** to send an alert to the confirmed email subscription.
8. Provided an alarm name for the CPU utilization alarm.
9. Created the CloudWatch alarm.
10. Used the stress command on the EC2 instance to increase CPU utilization above 70%.
11. Verified that the CloudWatch alarm changed to **ALARM** state.
12. Received the **email alert** successfully.





Result

The CloudWatch CPU utilization alarm was successfully triggered when the EC2 CPU utilization exceeded 70%, and an email notification was received successfully.

Conclusion

Successfully created and tested a CloudWatch alarm for EC2 CPU utilization above 70%. The alarm successfully detected high CPU utilization and sent an email notification through SNS.

5. Create a Dashboard and monitor the Tomcat service whether it is running or not and send the alert.

Objective

To monitor the Tomcat service using Amazon CloudWatch, create a custom metric, and prepare it for service-status monitoring.

Steps Performed

1. Installed Java 17

```
java -version
```

Verified Java 17 was installed.

2. Downloaded and installed Apache Tomcat 10.1.59

```
sudo mkdir -p /opt/tomcat
sudo tar -xzf /tmp/apache-tomcat-10.1.59.tar.gz -C /opt/tomcat --strip-components=1
sudo chown -R tomcat:tomcat /opt/tomcat
```

3. Created a Tomcat system user

```
sudo useradd -r -m -U -d /opt/tomcat -s /bin/false tomcat
```

4. Created a systemd service for Tomcat

Created:

```
/etc/systemd/system/tomcat.service
```

Then executed:

```
sudo systemctl daemon-reload
sudo systemctl enable --now tomcat
sudo systemctl status tomcat
```

Verified that Tomcat was running successfully.

5. Created an IAM role for CloudWatch

Created and attached:

IAM Role: EC2-Tomcat-CloudWatch-Role-asif-2026

Policy: CloudWatchAgentServerPolicy

Attached the role to the EC2 instance.

6. Created a custom CloudWatch metric

Namespace:

Custom/Tomcat

Metric:

TomcatStatus

Status values:

1 = Tomcat Running

0 = Tomcat Stopped

Instance ID:

i-0aba8afdcf9a3a884

7. Created the Tomcat monitoring script

Created:

/usr/local/bin/tomcat-status.sh

The script checks the Tomcat service using:

systemctl is-active --quiet tomcat

and publishes the status to CloudWatch using:

aws cloudwatch put-metric-data

8. Tested the CloudWatch metric

The metric was successfully published and verified in:

CloudWatch → Metrics → Custom/Tomcat → TomcatStatus

The metric showed:

TomcatStatus = 1

This confirmed that Tomcat is running and CloudWatch is receiving the custom metric.

```
[ec2-user@ip-10-0-1-208 tmp]$ sudo systemctl status tomcat
● tomcat.service - Apache Tomcat
   Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 10:30:22 UTC; 11s ago
     Process: 27466 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 27473 (java)
      Tasks: 30 (limit: 1059)
     Memory: 70.4M
        CPU: 3.846s
      CGroup: /system.slice/tomcat.service
              └─27473 /usr/lib/jvm/java-17-amazon-corretto.x86_64/bin/java -Djava.util.logging.config.file=/opt/tomcat/conf/logging.properties -Djava.util.logging.manager=

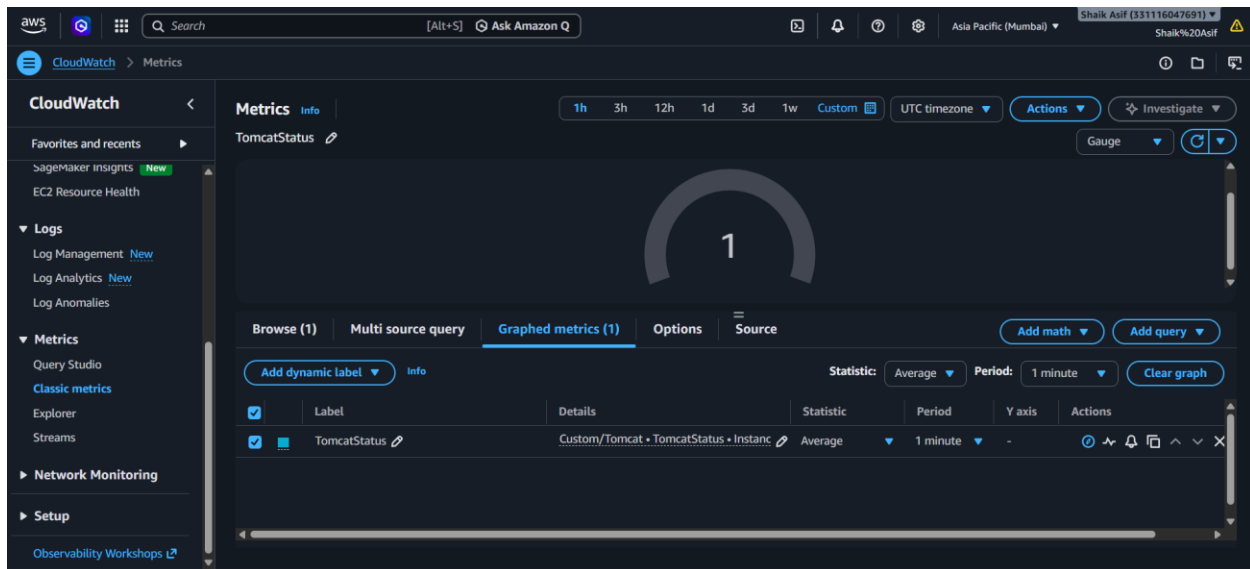
Sep 10 10:30:22 ip-10-0-1-208.ap-south-1.compute.internal systemd[1]: Starting tomcat.service - Apache Tomcat...
Sep 10 10:30:22 ip-10-0-1-208.ap-south-1.compute.internal startup.sh[27466]: Tomcat started.
Sep 10 10:30:22 ip-10-0-1-208.ap-south-1.compute.internal systemd[1]: Started tomcat.service - Apache Tomcat.
```

The screenshot shows the AWS IAM console interface. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, Access reports, and Images. The main content area displays the details for the role **EC2-Tomcat-CloudWatch-Role-asif-2026**. The role's summary includes its creation date (September 10, 2026), ARN, and instance profile ARN. The permissions tab is selected, showing a list of permissions policies attached to the role, including **CloudWatchAgentServerPolicy**. The permissions boundary is noted as not set.

Policy name	Type	Attached entities
CloudWatchAgentServerPolicy	AWS managed	1

The screenshot shows the AWS EC2 console interface. The left sidebar contains navigation links for EC2, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, Capacity Manager, Images, and Elastic Block Store. The main content area displays the details for the instance **i-0aba8afdcf9a3a884**. The instance's summary includes its instance ID, public IPv4 address, instance state (Running), hostname type, answer private resource DNS name, auto-assigned IP address, IAM role, IMDSv2 status, operator, public IPv4 address, instance state, private IP DNS name, instance type, VPC ID, subnet ID, instance ARN, private IPv4 addresses, public DNS, elastic IP addresses, AWS Compute Optimizer finding, and auto scaling group name.

Instance ID	Public IPv4 address	Private IPv4 addresses
i-0aba8afdcf9a3a884	65.2.11.182 open address	10.0.1.208



Conclusion

Tomcat was successfully installed and configured as a systemd service. An IAM role was attached to the EC2 instance, and a custom TomcatStatus CloudWatch metric was created. The metric successfully reported 1, confirming that the Tomcat service is running and its status can be monitored through CloudWatch.

6. Create a Dashboard and monitor the Nginx service to send the alert if Nginx is not running.

Objective

To monitor the Nginx service using Amazon CloudWatch Dashboard and send an alert when Nginx is not running.

Steps Performed

1. Installed Nginx

```
sudo dnf install nginx -y
```

2. Started and enabled Nginx

```
sudo systemctl enable --now nginx
```

Verified Nginx was active (running).

3. Created Nginx monitoring script

Created:

```
/usr/local/bin/nginx-status.sh
```

The script checks the Nginx service status and sends the result to CloudWatch.

1 = Nginx running

0 = Nginx not running

CloudWatch details:

Namespace: Custom/Nginx

Metric: NginxStatus

Instance ID: i-0aba8afdcf9a3a884

4. Made the script executable and tested it

```
sudo chmod +x /usr/local/bin/nginx-status.sh
```

```
sudo /usr/local/bin/nginx-status.sh
```

The script executed successfully without errors.

5. Configured automatic monitoring

Created a systemd service:

```
/etc/systemd/system/nginx-monitor.service
```

Created a systemd timer:

```
/etc/systemd/system/nginx-monitor.timer
```

Configured the timer to execute the monitoring script every minute.

6. Verified the CloudWatch metric

CloudWatch → Metrics → Custom/Nginx → NginxStatus

The metric showed:

NginxStatus = 1

confirming that Nginx was running and the metric was being received.

7. Created CloudWatch Dashboard

Dashboard name:

Nginx-Monitoring-Dashboard

Added the NginxStatus metric with a 1-minute period.

8. Created CloudWatch Alarm

Alarm name:

Nginx-Service-Down

```
[ec2-user@ip-10-0-1-208 ~]$ sudo dnf install nginx -y
Last metadata expiration check: 1:10:49 ago on Thu Sep 10 10:04:11 2026.
Dependencies resolved.

```

Package	Architecture	Version	Repository	Size
Installing:				
nginx	x86_64	1:1.30.4-1.amzn2023.0.1	amazonlinux	34 k
Installing dependencies:				
generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3	amazonlinux	19 k
gperftools-libs	x86_64	2.9.1-1.amzn2023.0.3	amazonlinux	308 k
libunwind	x86_64	1.4.0-5.amzn2023.0.3	amazonlinux	66 k
nginx-core	x86_64	1:1.30.4-1.amzn2023.0.1	amazonlinux	711 k
nginx-filesystem	noarch	1:1.30.4-1.amzn2023.0.1	amazonlinux	10 k
nginx-mimetypes	noarch	2.1.49-3.amzn2023.0.3	amazonlinux	21 k

```
Transaction Summary
Install 7 Packages

Total download size: 1.1 M
Installed size: 3.8 M
Downloading Packages:
(1/7): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm 490 kB/s | 19 kB 00:00
(2/7): libunwind-1.4.0-5.amzn2023.0.3.x86_64.rpm 1.5 MB/s | 66 kB 00:00
(3/7): gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm 6.1 MB/s | 308 kB 00:00
```

```
[ec2-user@ip-10-0-1-208 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 11:15:27 UTC; 8s ago
     Process: 30418 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 30419 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 30420 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 30421 (nginx)
     Tasks: 3 (limit: 1059)
    Memory: 3.7M
       CPU: 56ms
   CGroup: /system.slice/nginx.service
           └─30421 "nginx: master process /usr/sbin/nginx"
             └─30422 "nginx: worker process"
               └─30423 "nginx: worker process"

Sep 10 11:15:27 ip-10-0-1-208.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 10 11:15:27 ip-10-0-1-208.ap-south-1.compute.internal nginx[30419]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 10 11:15:27 ip-10-0-1-208.ap-south-1.compute.internal nginx[30419]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 10 11:15:27 ip-10-0-1-208.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```

CloudWatch Metrics

Navigation: CloudWatch > Metrics

Time range: 1h 3h 12h 1d 3d 1w Custom UTC timezone Actions Investigate

Alerts: You have to set the gauge range

Visualizer: Gauge (Value: 1)

Options: Browse (1) Multi source query Graphed metrics (1/2) Options Source Add math Add query

Search: Search for any metric, dimension, resource id or account id

Instance name 1/1	Instancelid	Metric name	Alarms
fdghjk	i-0aba8afdcf9a3a...	NginxStatus	No alarms

CloudWatch Dashboards > Nginx-Monitoring-Dashboard

Autosave: Off

Time range: 1h 3h 12h 1d 3d 1w Custom UTC timezone Actions Save

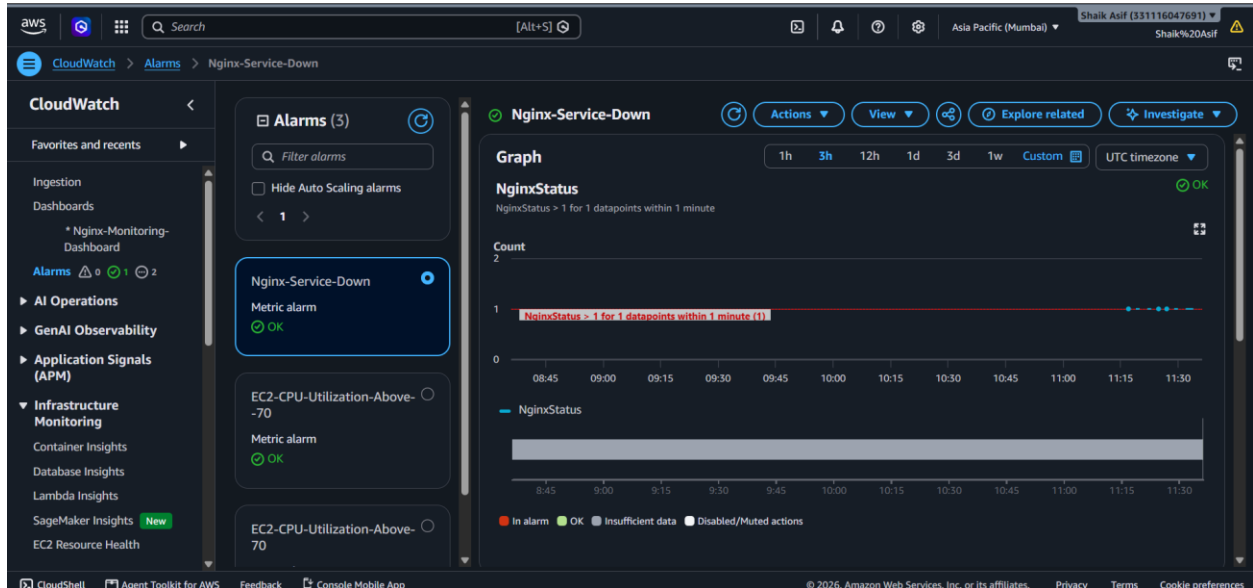
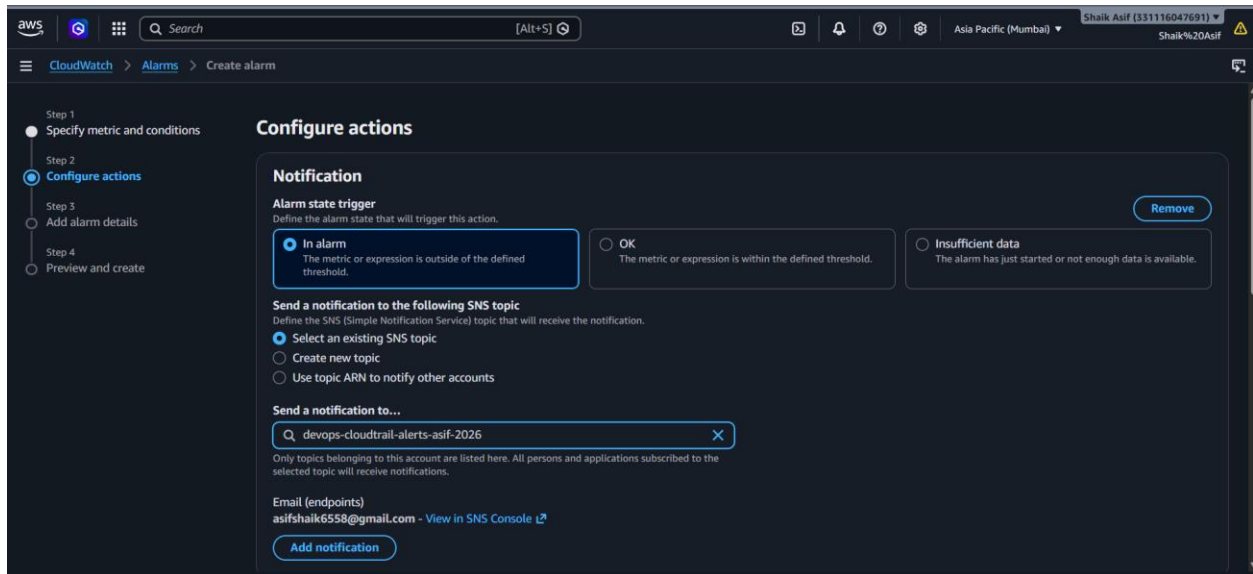
Widget: NginxStatus Count 2

Y-axis: 0 1 2

X-axis: 10:30 10:45 11:00 11:15

Legend: NginxStatus

Footer: CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Conclusion

Nginx was successfully installed and configured for CloudWatch monitoring. A custom **NginxStatus** metric, CloudWatch dashboard, and alarm were created. The monitoring system checks Nginx every minute and sends an **SNS email alert when Nginx is not running**.